

TOC (BCS503)

IA 1 - QB Answers

1. Define the following terms with example.

(a) String

- **Definition:** A string (or word) is a finite sequence of symbols chosen from a specific alphabet.
- **Example:** If we have the binary alphabet $\Sigma = \{0, 1\}$, a valid string could be **01101**. Here, each symbol (0 or 1) is chosen from Σ and combined in sequence to form the string.

(b) Alphabet (Σ)

- **Definition:** An alphabet is a finite, non-empty set of symbols from which strings are formed. It's usually denoted by the symbol Σ .
- **Example:**
 - **Binary alphabet:** $\Sigma = \{0, 1\}$.
 - **Lower-case alphabet:** $\Sigma = \{a, b, \dots, z\}$.
 - **Custom example:** $\Sigma = \{x, y, z\}$.

(c) Language

- **Definition:** A language is a set of strings formed using symbols from a given alphabet (Σ). It includes various sequences of symbols that adhere to specific patterns or rules.
- **Examples:**
 1. **Language of equal numbers of 0's and 1's:** $\{\epsilon, 01, 10, 0011, 0101, 1001, \dots\}$.
 2. **Binary language with n 0's followed by n 1's:** $\{\epsilon, 01, 0011, 000111, \dots\}$.
 3. **Language of binary prime numbers:** $\{10, 11, 101, 111, 1011, \dots\}$.

(d) Deterministic Finite Automata (DFA)

- **Definition:** A DFA is a finite-state machine where each input symbol leads to exactly one state transition. The term "deterministic" indicates no ambiguity in state transitions.
- **Structure of DFA:** Represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:
 - **Q:** Finite set of states.
 - **Σ :** Finite input alphabet.
 - **δ (transition function):** Maps a state and input symbol to a single state ($\delta: Q \times \Sigma \rightarrow Q$).
 - **q_0 :** Start state.
 - **F:** Set of accepting (final) states.
- **Example:** A DFA that accepts binary strings ending in "01" will only transition to an accepting state if the last two characters are "01".

(d) Deterministic Finite Automata (DFA)

- **Definition:** A DFA is a finite-state machine where each input symbol leads to exactly one state transition. The term "deterministic" indicates no ambiguity in state transitions.
- **Structure of DFA:** Represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:
 - **Q:** Finite set of states.
 - **Σ :** Finite input alphabet.

- **δ (transition function):** Maps a state and input symbol to a single state ($\delta: Q \times \Sigma \rightarrow Q$).
 - **q_0 :** Start state.
 - **F :** Set of accepting (final) states.
- **Example:** A DFA that accepts binary strings ending in "01" will only transition to an accepting state if the last two characters are "01".

(e) Nondeterministic Finite Automata (NFA)

- **Definition:** An NFA is a finite-state machine where multiple transitions can exist for a single input symbol from a given state. An NFA can be in multiple states simultaneously.
- **Structure of NFA:** Also represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, but δ can map to zero, one, or more states for each input.
- **Example:** An NFA that accepts strings ending in "01" could explore multiple state paths to reach an accepting state if the input string ends in "01".

(f) ϵ -NFA (Epsilon-NFA)

- **Definition:** An ϵ -NFA is a nondeterministic finite automaton that includes epsilon (ϵ) transitions, allowing the automaton to transition between states without consuming any input symbols.
- **Structure:** Represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:
 - **Q :** Finite set of states.
 - **Σ :** Input alphabet.
 - **δ :** Transition function, mapping states and inputs (including ϵ) to subsets of states, $\delta: Q \times \{\Sigma \cup \epsilon\} \rightarrow 2^Q$.
 - **q_0 :** Start state.
 - **F :** Set of accepting states.
- **Example:** In an ϵ -NFA, the automaton could move from state q to state r just with an ϵ transition, without needing any actual input symbol.

(g) ϵ -Closure

- **Definition:** The ϵ -closure of a state in an NFA is the set of all states that can be reached from the given state using only ϵ transitions.
- **Properties:**
 1. **Every state** in the set (S) is included in its own ϵ -closure.
 2. **Any state reachable by ϵ transitions** from any state in S is included in $\epsilon(S)$.
 3. **No other states** are included in $\epsilon(S)$ beyond those reachable by ϵ .
- **Example:**
 - If state q can reach state s via an ϵ transition, then $\epsilon\text{-closure}(q) = \{q, s\}$.

(h) Powers of an Alphabet

- **Definition:** If Σ is an alphabet, then the set of all strings of a certain length k from that alphabet is denoted as Σ^k .
- **Examples:**
 - Σ^0 : Contains only the empty string, $\{\epsilon\}$.
 - Σ^1 : If $\Sigma = \{0, 1\}$, then $\Sigma^1 = \{0, 1\}$.
 - Σ^2 : For $\Sigma = \{0, 1\}$, $\Sigma^2 = \{00, 01, 10, 11\}$.
 - Σ^* : The set of all strings of any length from Σ , including the empty string (e.g., $\Sigma = \{0, 1\}$ results in $\{\epsilon, 0, 1, 00, 01, \dots\}$).

(i) Language of DFA

- **Definition:** The language of a Deterministic Finite Automaton (DFA) is the set of all strings it accepts. This includes only those strings for which the DFA reaches an accepting (final) state.
- **Example:** A DFA designed to accept strings with an even number of 1s will have a language such as $L = \{\epsilon, 00, 11, 0110, \dots\}$.

(j) Language of NFA

- **Definition:** The language of a Nondeterministic Finite Automaton (NFA) is the set of all strings for which the NFA can reach an accepting state. Unlike a DFA, an NFA may have multiple paths and can use epsilon transitions.
- **Example:** An NFA accepting strings that end in "01" will accept strings like $L = \{01, 101, 1101, \dots\}$.

2. Explain the differences between DFA and NFA.

Ans:

Difference between DFA, NFA, ϵ -NFA

DFA	NFA	ϵ -NFA
DFA is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where Q is non-empty, finite set of states. Σ is non-empty, finite set of input alphabets. $\delta : Q \times \Sigma \rightarrow Q$. $q_0 \in Q$ is the start state. $F \subseteq Q$ is set of accepting or final states.	$M = (Q, \Sigma, \delta, q_0, F)$ where Q is non empty, finite set of states. Σ is non empty, finite set of input alphabets. $\delta : Q \times \Sigma \rightarrow 2^Q$. $q_0 \in Q$ is the start state. $F \subseteq Q$ is set of accepting or final states	$M = (Q, \Sigma, \delta, q_0, F)$ where Q is non empty, finite set of states. Σ is non empty, finite set of input alphabets $\delta : \text{from } Q \times \{\Sigma \cup \epsilon\} \rightarrow 2^Q$. $q_0 \in Q$ is the start state. $F \subseteq Q$ is set of accepting or final states.
There can be zero or one transition from a state on an input symbol;	There can be zero, one or more number of transitions from a state on an input symbol	There can be zero, one or more number of transitions from a state with or without an input symbol
Difficult to design	The NFA are easier to design	Easy to construct using regular expression
More number of transitions	Less number of transitions	More number of transitions compared to NFA
Less powerful since at any point of time it will be in only one state	More powerful; than DFA since it can be in more than one state	More powerful than NFA since at any point of time it will be in more than one state with or without giving any input.

Feature	DFA (Deterministic Finite Automaton)	NFA (Nondeterministic Finite Automaton)
Definition	Each input symbol leads to exactly one state.	An input symbol can lead to multiple states or none at all.
Transition Function	Each state and input symbol have one unique next state.	Each state and input symbol may have zero, one, or multiple next states.
Epsilon (ϵ) Transitions	Does not allow ϵ -transitions (no input transitions).	Allows ϵ -transitions, enabling state changes without input.
Acceptance of Strings	Accepts a string if it reaches an accepting state following a unique path.	Accepts a string if any path reaches an accepting state.
Complexity	Typically requires more states to handle all paths.	Often has fewer states due to branching flexibility.
Memory Consumption	More memory consumption due to the unique state for each transition.	Less memory consumption due to fewer defined transitions and states.
Equivalence	Any NFA can be converted to an equivalent DFA.	Recognizes the same languages as DFA (regular languages).
Usage	Easier for hardware/software implementation, used in real-time applications.	Preferred for complex languages in theory and proofs.

3. Prove that,

Theorem 2.11: If $D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$ is the DFA constructed from NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$ by the subset construction, then $L(D) = L(N)$.

Ans:

Theorem: If $M_D = (Q_D, \Sigma_D, \delta_D, q_0, F_D)$ is the DFA constructed from NFA $M_N = (Q_N, \Sigma, \delta_N, q_0, F_N)$ by the subset construction, then $L(M_D) = L(M_N)$.

Proof: Let $|w|=0$, that is $w=\epsilon$. By the basis definitions of δ^* for DFA's and NFA's both $\delta^*(\{q_0\}, \epsilon)$ and $\delta^*(q_0, \epsilon)$ are $\{q_0\}$

Let w be of length $n+1$, and assume the statement for length n . break w as $w=xa$, where a is the final symbol of w . by the inductive hypothesis $\delta^*(\{q_0\}, x) = \delta^*(q_0, x)$. let both these sets of N's states be $\{p_1, p_2, \dots, p_k\}$

The inductive part of the definition of δ^* for NFA's tells that:

$$\delta^*(q_0, w) = \bigcup_{i=1}^k \delta_N(p_i, a) \text{ -----1}$$

The subset construction, on the other hand, tells that :

$$\delta_D(\{p_1, p_2, \dots, p_k\}, a) = \bigcup_{i=1}^k \delta_N(p_i, a) \text{ -----2}$$

Using eqn 2 and the fact that $\delta^*(\{q_0\}, x) = \{p_1, p_2, \dots, p_k\}$ in the inductive part of the definition of δ^* for DFA's

Theorem:

If $M_D = (Q_D, \Sigma, \delta_D, q_0, F_D)$ is the DFA constructed from the NFA $M_N = (Q_N, \Sigma, \delta_N, q_0, F_N)$ by the subset construction, then $L(M_D) = L(M_N)$.

Proof:

Let $|w| = 0$, that is $w = \epsilon$. By the basis definitions of δ^* for both DFAs and NFAs, we have $\delta^*(\{q_0\}, \epsilon) = \delta^*(q_0, \epsilon) = \{q_0\}$.

Now, let w be of length $n + 1$, and assume the statement holds for length n . Break w as $w = xa$, where a is the final symbol of w . By the inductive hypothesis, $\delta^*(\{q_0\}, x) = \delta^*(q_0, x)$. Let this set of NFA states be $\{p_1, p_2, \dots, p_k\}$.

The inductive part of the definition of δ^* for NFAs tells us that:

$$\delta^*(q_0, w) = \delta_N(p_i, a) \quad \text{for each } p_i. \quad (1)$$

The subset construction, on the other hand, tells us that:

$$\delta_D(\{p_1, p_2, \dots, p_k\}, a) = \delta_N(p_i, a) \quad \text{for each } p_i. \quad (2)$$

Using Equation (2) and the fact that $\delta^*(\{q_0\}, x) = \{p_1, p_2, \dots, p_k\}$ in the inductive part of the definition of δ^* for DFAs, we get:

$$\delta^*(\{q_0\}, w) = \delta_D(\delta^*(\{q_0\}, x), a) = \delta_D(\{p_1, p_2, \dots, p_k\}, a) = \delta_N(p_i, a). \quad (3)$$

Thus, Equations (2) and (3) demonstrate that $\delta^*(\{q_0\}, w) = \delta_N^*(\{q_0\}, w)$.

Hence, we have proved that $L(M_D) = L(M_N)$.

Theorem:

If $M_D = (Q_D, \Sigma, \delta_D, q_0, F_D)$ is the DFA constructed from an NFA $M_N = (Q_N, \Sigma, \delta_N, q_0, F_N)$ using the subset construction, then $L(M_D) = L(M_N)$.

In other words, the language recognized by the DFA M_D is the same as the language recognized by the NFA M_N .

Step-by-Step Proof:**1. Initial Setup (Basis):**

Consider an input string w .

- Let's start by examining the simplest case where w has a length of 0, which means $w = \epsilon$ (the empty string).
- According to the definition of δ^* (the extended transition function) for both DFAs and NFAs, applying δ^* with an empty string ϵ will return the start state. Thus:

$$\delta^*({q_0}, \epsilon) = \delta^*(q_0, \epsilon) = {q_0}.$$

This shows that both the DFA and the NFA start in their initial state when no input is given.

2. Inductive Hypothesis (Assumption):

Now, we assume that the theorem holds for strings of length n .

Specifically, we assume that for any string x of length n :

$$\delta^*({q_0}, x) = \delta^*(q_0, x).$$

This means that for any string of length n , the set of states reached by the NFA and the DFA is the same.

3. Inductive Step (String of Length $n + 1$):

Let's take a string w of length $n + 1$. We can break w into two parts: $w = xa$, where x is a string of length n and a is the last symbol of w .

4. Using the Inductive Hypothesis for x :

By our assumption (inductive hypothesis), we know that for the string x of length n :

$$\delta^*({q_0}, x) = \delta^*(q_0, x).$$

Let this result be a set of NFA states $\{p_1, p_2, \dots, p_k\}$ after processing x .

5. Handling the Transition for Symbol a :

Now, let's see what happens when we apply the symbol a to this set of states $\{p_1, p_2, \dots, p_k\}$.

- For the NFA M_N :

The NFA transition function δ_N might move from each state p_i (where i ranges from 1 to k) to multiple new states on input a . In general:

$$\delta^*(q_0, w) = \delta_N^*(\{p_1, p_2, \dots, p_k\}, a).$$

- For the DFA M_D :

In the DFA, we use the subset construction to ensure that M_D moves to the exact same set of states on input a :

$$\delta_D(\{p_1, p_2, \dots, p_k\}, a).$$

6. Conclusion of the Inductive Step:

We have shown that both M_D and M_N reach the same set of states for any string w of length $n + 1$, meaning:

$$\delta^*(\{q_0\}, w) = \delta_N^*(\{q_0\}, w).$$

Final Conclusion:

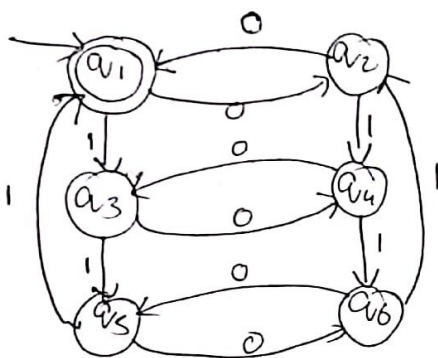
By mathematical induction, we've proven that for any string w , M_D and M_N process w identically, leading to the same resulting set of states. Hence, we conclude:

$$L(M_D) = L(M_N).$$

4. Build a DFA for each of the following languages.

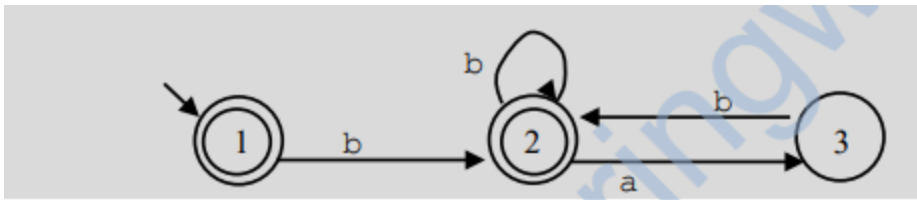
[For each solution also write: Language Input Accepted i.e $L = \{ab, aba, abb, \dots\}$ Also write Not Accepted $L' = \{\epsilon, a, b, \dots\}$

For where $\delta = \{ \delta(q_0, 0) = q_2, \delta(q_0, 1) = q_0, \delta(q_1, 0) = q_1, \delta(q_1, 1) = q_1, \delta(q_2, 0) = q_2, \delta(q_2, 1) = q_1 \}$

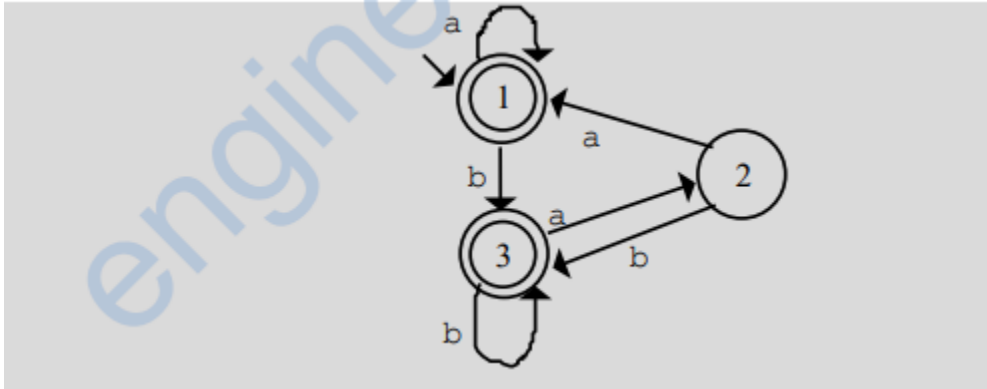


DFA $\{Q, \Sigma, S, q_0, F\}$.
 $Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$
 S as show in figure
 q_0 is initial state.
 q_1 is final state.
 $\Sigma = \{0, 1\}$

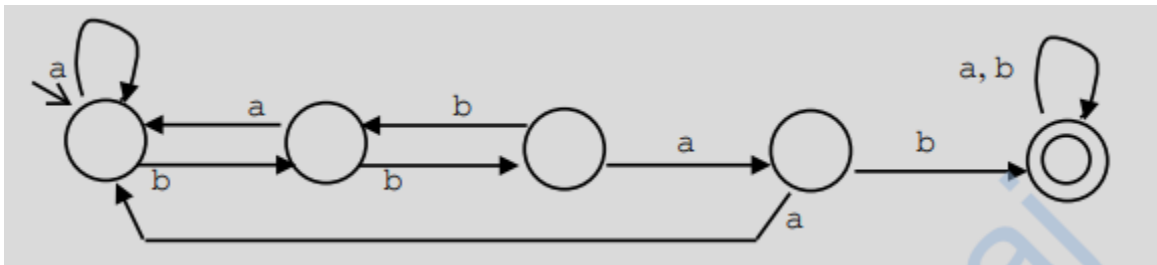
a) $L = \{w \in \{a, b\}^* : \text{every } a \text{ in } w \text{ is immediately preceded and followed by } b\}$.



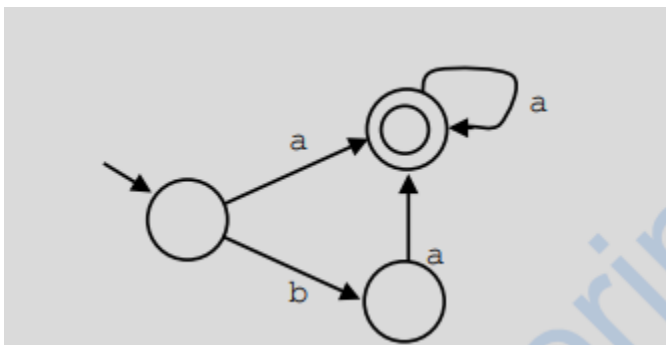
b) $L = \{w \in \{a, b\}^* : w \text{ does not end in } ba\}.$



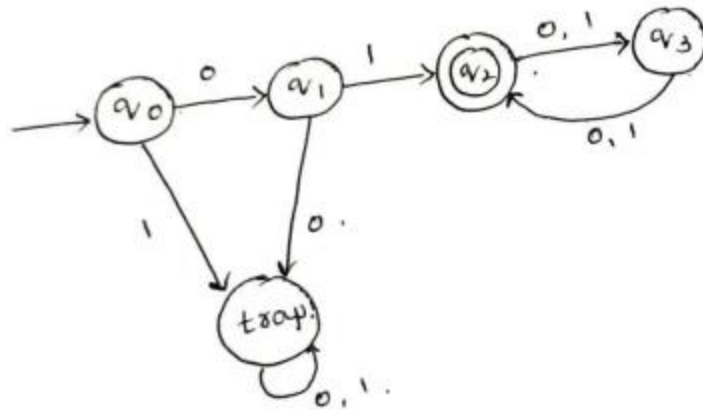
c) $L = \{w \in \{a, b\}^* : w \text{ has } bbab \text{ as a substring}\}.$



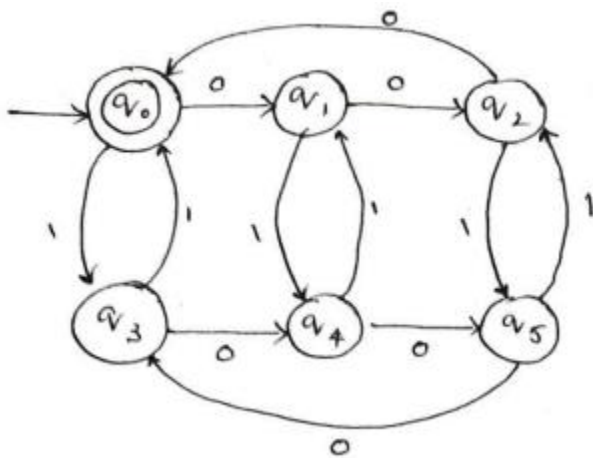
d) $L = \{w \in \{a, b\}^* : w \text{ has neither } ab \text{ nor } bb \text{ as a substring}\}.$



e) $L = \{w \in \{0, 1\}^* : w \text{ is of even length and begins with } 01\}.$



f) $L = \{w \in \{0, 1\}^* : \text{strings such that number of 1's is even and the number of 0's is a multiple of 3}\}$

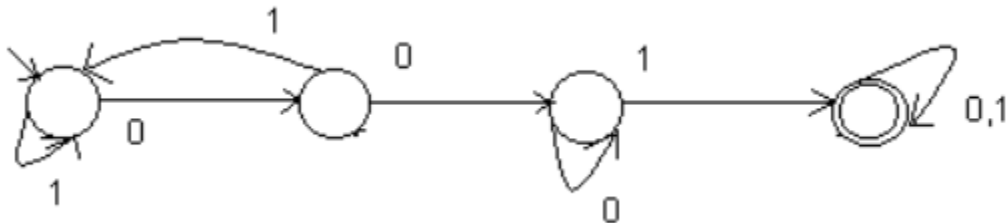


$$n_0(w) \bmod 3 = 0$$

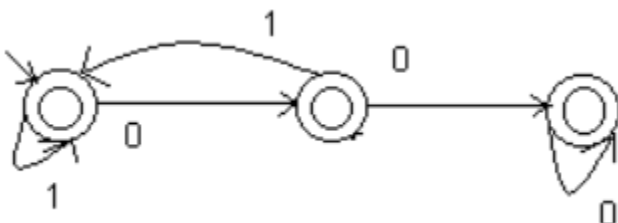
$$n_1(w) \bmod 2 = 0$$

g) $L = \{w \in \{0, 1\}^* : w \text{ corresponds to the binary encoding, without leading 0's, of natural numbers that are evenly divisible by 3}\}.$

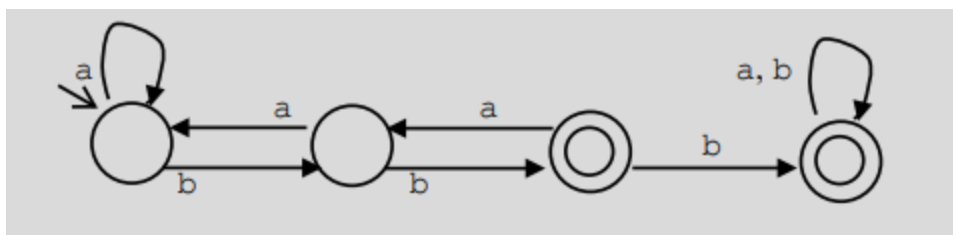
h) $L = \{w \in \{0, 1\}^* : w \text{ has 001 as a substring}\}$



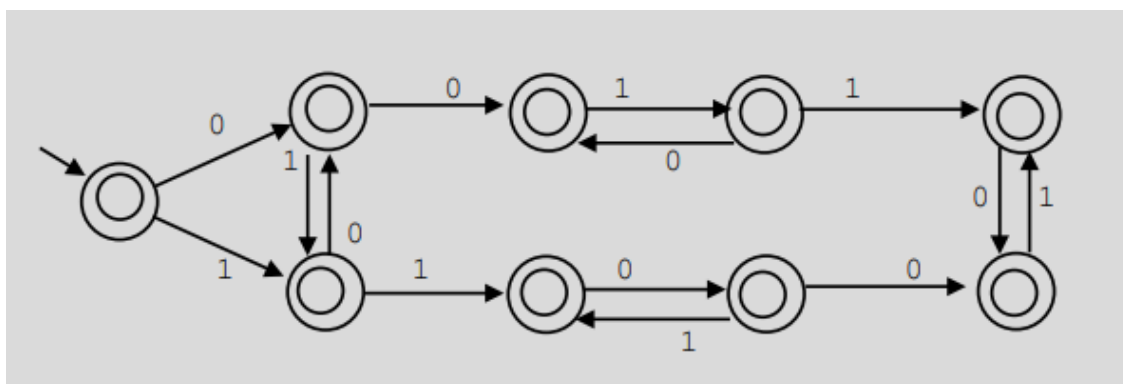
i) $L = \{w \in \{0, 1\}^* : w \text{ does not have 001 as a substring}\}.$



j) $L = \{w \in \{a, b\}^* : w \text{ contains at least two b's that are not immediately followed by a's}\}.$



k) The set of binary strings with at most one pair of consecutive 0's and at most one pair of consecutive 1's.



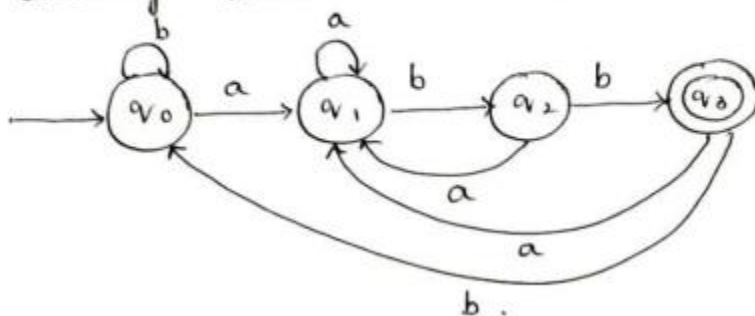
l) $L = \{w \in \{a, b\}^* : |w| \bmod 5 = 0\}$

m) $L = \{w \in \{a, b\}^* : \text{In } w, 4$

th character from last is a }

n) $L = \{w \in \{a, b\}^* : w \text{ is not ending with } abb\}$.

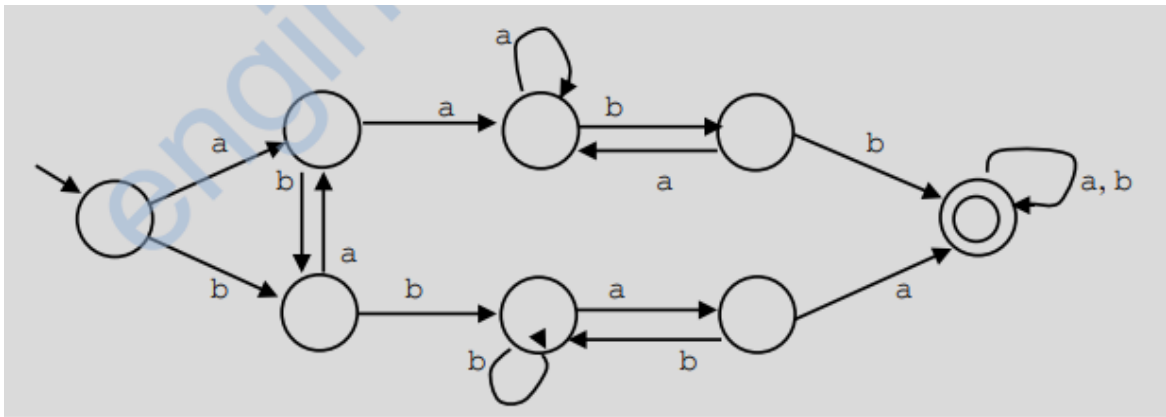
4. Draw a dfa to accept the strings of a & b ending with abb. $\Sigma = \{a, b\}$.



o) $L = \{w \in \{a, b\}^* : \text{all strings with at least one 'a' and exactly two 'b's}\}$

p) $L = \{w \in \{a-z\}^* : \text{all five vowels occur in } w \text{ in alphabetical order}\}$

q) $L = \{w \in \{a, b\}^* : w \text{ has both } aa \text{ and } bb \text{ as a substrings}\}$.



5. Convert following NFA to DFA using subset construction method

a)